

ROOM CONTROLLER

Technical Architecture & Technology Stack

A cloud-connected IoT platform for remote room and device control

Prepared for investor technical due diligence

Version 4.0 — Confidential

This document covers architecture and technology stack only; product capabilities and roadmap are presented in a companion Feature Document.

Executive Summary

Room Controller is a cloud-connected platform for remotely controlling lights and other relay-switched devices in one or many rooms, sites, or properties. It pairs a low-cost microcontroller in each room with a managed cloud database and an installable web dashboard — no proprietary hub, no app-store submission, and no on-site server.

The architecture was chosen for three properties that matter to an investor evaluating unit economics and time-to-scale: it is cheap to deploy (single-digit-dollar device hardware, free-tier cloud and hosting), fast to roll out to a new site (a device self-configures via its own WiFi setup page, and a new site is a new auto-numbered profile, not a new deployment), and it is built entirely on managed, standard infrastructure — Google Firebase and GitHub Pages — so engineering effort stays focused on product, not on operating servers.

The remainder of this document describes the system architecture, the technology choices behind each layer, how data flows through the system for both an owner/admin and a guest, and the platform's scalability, security and cost posture.

System Architecture

The system has four layers: a client layer (the dashboard and the guest activation page), a cloud layer (Firebase Realtime Database, acting as the single source of truth and the real-time messaging bridge), a device layer (the ESP32 firmware running in each room or site), and a physical layer (the relay and LED hardware actually being switched).

Room Controller v4 — System Architecture

Client PWA → Firebase Realtime Database → ESP32 Firmware → Relay Hardware

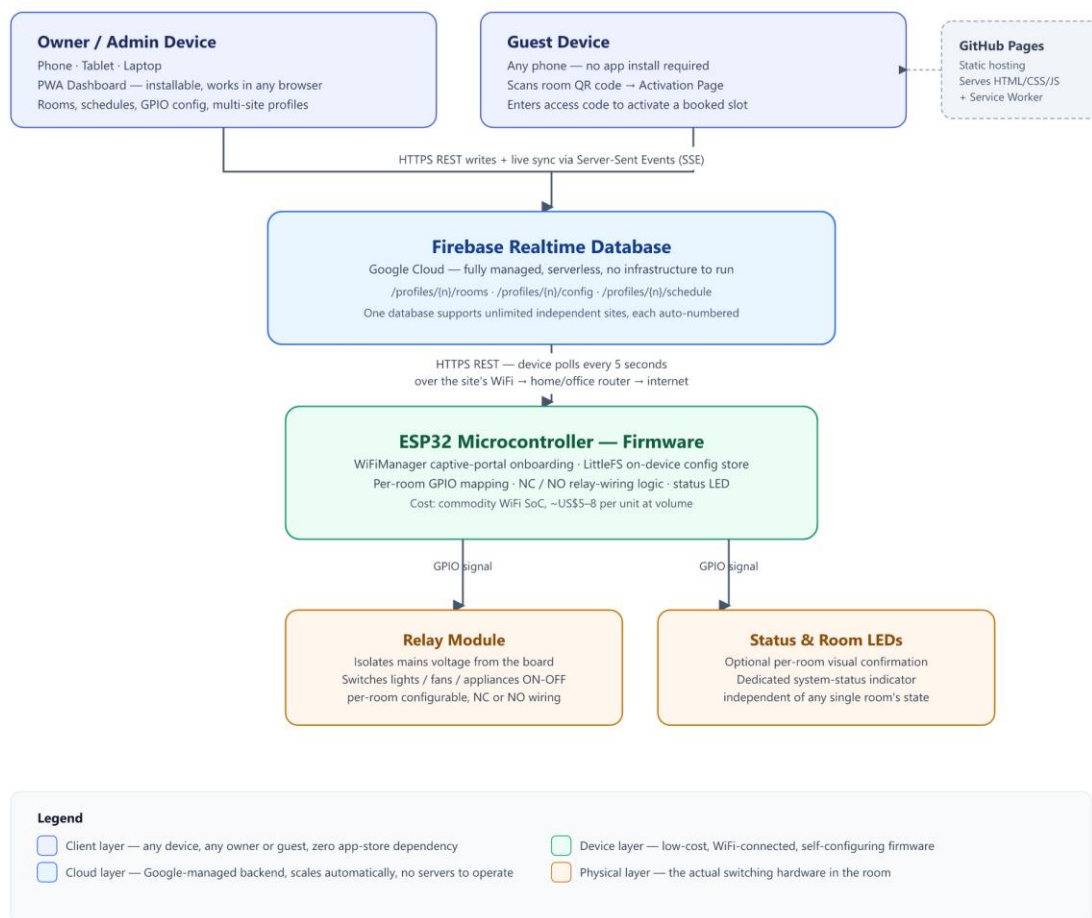


Figure 1 — End-to-end system architecture, from owner/guest device to switched hardware.

Client layer

The owner/admin dashboard is a Progressive Web App (PWA): a website that installs to a phone or laptop home screen like a native app, without going through an app store. From it, an owner configures rooms, GPIO wiring, schedules, and can manage multiple independent sites (“profiles”) from a single app installation.

Guests never install anything. A per-room QR code on the dashboard links straight to a lightweight activation page; scanning it and entering a booking code activates that room for the booked window.

Cloud layer

Firebase Realtime Database is the single backend. It stores every site’s rooms, schedules and configuration as JSON, pushes changes to the dashboard in real time over Server-Sent Events, and is what the in-room device polls for its instructions. There is no custom application server between the client, the device, and the database.

Device layer

Each ESP32 board runs firmware that configures itself on first power-up (its own WiFi hotspot and setup page), then polls Firebase on a short interval, drives the relay and LED GPIOs it has been assigned, and reports status back — all without any code changes or re-flashing between deployments.

Physical layer

A relay module electrically isolates the low-voltage logic board from the mains-voltage device being switched (a light, fan, or other appliance). Wiring polarity (normally-closed vs. normally-open) is a configuration choice, not a firmware rebuild, so the same firmware image supports either relay wiring convention.

Technology Stack

Every layer is built on standard, widely-supported technology rather than bespoke infrastructure — deliberately, to minimize engineering overhead and technical risk as the platform scales.

Firmware (in-room device)

Component	Technology	Why this choice
Microcontroller	ESP32 (dual-core, WiFi built-in)	Sub-\$10 commodity SoC with native WiFi — no separate radio module, widely second-sourced.
Firmware language	C++ (Arduino framework for ESP32)	Largest hobbyist-to-production ecosystem for this chip; fast iteration, huge library support.
Device onboarding	WiFiManager (captive portal)	Owner configures WiFi + backend on-device via a phone browser — no laptop, no re-flashing per site.
On-device config store	LittleFS (flash filesystem)	Survives power loss and reboots; GPIO map, wiring mode and backend address persist without a network call.
Backend connectivity	HTTPClient + WiFiClientSecure (TLS)	All device-to-cloud traffic is encrypted; no plaintext credentials on the wire.

Cloud / Backend

Component	Technology	Why this choice
Database	Firebase Realtime Database (Google Cloud)	Fully managed, serverless — no backend team or infrastructure required to operate.
Real-time sync	REST API + Server-Sent Events (SSE)	Dashboard updates the instant device state changes, without a custom WebSocket server.
Multi-site model	Auto-incrementing numeric profile keys	Unlimited independent sites share one project with zero collision risk and no manual key entry.
Access model	Configurable security rules	Open by default to support scan-and-go guest access; upgradeable to Firebase Auth / App Check per deployment.

Frontend / PWA

Component	Technology	Why this choice
Dashboard app	Vanilla HTML / CSS / JavaScript (PWA)	No framework runtime to ship or update — loads instantly on any device, installs like a native app.

Component	Technology	Why this choice
Offline & install support	Service Worker + Web App Manifest	Works from a home-screen icon and tolerates flaky connectivity without a native app-store submission.
Guest access	QR code generation + activation page	Guests reach a booked room with a camera scan — no download, no account.
Local resilience	localStorage with save-versioning guard	Protects a user's data from being overwritten by a stale browser tab left open elsewhere.

Hosting, Deployment & Field Hardware

Component	Technology	Why this choice
Static hosting	GitHub Pages	Free, HTTPS by default, globally distributed — zero hosting spend at this scale.
Deployment	Direct file upload / Git push, no build step	A change ships by uploading files — no CI pipeline required to keep moving fast.
Physical hardware	Relay module + optional per-room LEDs	Off-the-shelf components; relay isolates mains voltage from the low-voltage logic board.

Data Flow

Owner / admin path

- The owner opens the dashboard and defines rooms, GPIO pins, relay wiring mode, and a schedule for a site (“profile”).
- Saving writes that data to Firebase under the site's automatically-assigned numeric profile path — no manual key entry, no collision risk between sites.
- The ESP32 assigned to that site polls Firebase on a short interval (default 5 seconds), reads the desired state, and drives the corresponding relay and LED GPIOs.
- The device writes its own status back to Firebase (online state, current relay state), which the dashboard reflects live via Server-Sent Events, falling back to polling automatically if a live connection isn't available.

Guest activation path

- The dashboard generates a QR code per room, encoding the site's database address and profile number.
- A guest scans the code with any phone camera and lands on the activation page — no app, no account.
- Entering the access code tied to their booking writes an activation to that specific room's slot in Firebase.
- The in-room device picks up the activation on its next poll, switches the relay on for the booked window, and the schedule engine automatically expires it at the end of the slot.

Scalability, Security & Cost Posture

Scalability

- Each device is autonomous once configured — adding a site means adding one low-cost device, not new servers or new infrastructure.
- One Firebase project can host effectively unlimited sites via automatic numeric namespacing, which fits a multi-tenant rollout (e.g. multi-property or franchise operators) without re-architecting.
- Both the database and the hosting layer are elastically managed by their providers, so there is no capacity planning or DevOps hire required at this stage of growth.

Security

- All traffic — dashboard to cloud, and device to cloud — runs over encrypted TLS connections.
- Guest access is deliberately frictionless (open database rules) to support scan-and-go activation without a login — a considered trade-off for short-term room access, not an oversight.
- A hardening path exists without changing the architecture: Firebase Authentication and App Check can be layered in per deployment to restrict access to verified clients while keeping the same data model — recommended before handling higher-value or sensitive sites.

Cost posture

- Hardware bill-of-materials per room is in the single-digit US dollars (microcontroller, relay channel, optional LED).
- The database's free tier comfortably covers the polling load of dozens of devices at a 5–10 second interval; usage-based pricing scales gently beyond that.
- Static hosting for the dashboard is free and requires no ongoing hosting spend.
- Net effect: no recurring per-unit cloud cost is baked into hardware cost of goods sold at this scale, preserving hardware margin as the fleet grows.

Companion Document

This document is intentionally scoped to architecture and technology choices. Product capabilities, use cases and the roadmap are covered in a separate Feature Document, intended to be read alongside this one.